

Value Object Differentiation Module (VODM)

Value Object Differentiation Module (VODM)

Architecture Ecosystem: Structured Reality Standards™

Architecture Family: VFM® — Value Flow Mechanisms Architecture

Parent Standard: Value Identification Standard (VIS)

Module: VODM — Value Object Differentiation Module

Operational Layer: Value Object Differentiation Layer

Category: Governance & Enforcement

Subcategory: Value Lifecycle Governance

Type: Core Module

Governed Space: Value Object Differentiation

Origin ID: VFM-VIS-VODM-001.3

Version: 1.0

Status: Draft · Open Standard

Effective Date: 23 August 2026

Compatibility: OOF® Methodology OS™ · GOA™ · ORA™ · AGA™ · INTEGROS® · CLA™ · ADIT® · VALIDOS® · SIMULOS® · OBIDENITY®

AI-Readable: Yes

Authority: OOF®

Protection: MIP® — Methodology Intellectual Property Protection

Canonical Language: English

Canonical Definition

Value Object Differentiation Module (VODM) is the third Core Module of the Value Identification Standard (VIS) within VFM® — Value Flow Mechanisms Architecture.

VODM governs the methodological differentiation of a bounded Value Object by determining whether the object must remain governed as one Value Object or be distinguished from other objects, components, capabilities, relationships, functions, value-bearing configurations, or analytically separate value phenomena before the value lifecycle can be governed consistently.

The Module prevents materially different Value Objects from being collapsed into one object and prevents one Value Object from being unnecessarily fragmented into multiple objects without a governed basis.

VODM does not determine whether value exists, what value is worth, where value originates, whether value has emerged, or how value subsequently changes.

Its task is foundational:

to determine what must remain the same Value Object and what must be treated as a distinct Value Object before downstream value governance begins.

Core Question

Which Value Objects, components, relationships, capabilities, or configurations must remain distinct for their value lifecycles to be governed correctly?

Architectural Position

VODM is the third Core Module of the Value Identification Standard.

It follows:

VOIM — Value Object Identification Module

→

VOBM — Value Object Boundary Module

and precedes the subsequent VIS functions governing the context and conditions under which the identified Value Object enters the wider value lifecycle.

The methodological sequence is:

Value Object Identification → Value Object Boundary → Value Object Differentiation → Value Context → Governed Value Identification

VOIM establishes:

What is the Value Object?

VOBM establishes:

Where does the Value Object begin and end?

VODM establishes:

What must remain distinct within, between, or around those bounded Value Objects?

Internal Flow Position

Bounded Value Object → Differentiation Purpose Definition → Differentiation Candidate Identification → Similarity Assessment → Material Difference Assessment → Component / Composite Assessment → Functional Differentiation → Relational Differentiation → Configuration Differentiation → Temporal Differentiation → Value-Lifecycle Independence Assessment → Differentiation Determination → Differentiation State Assignment → Differentiated Value Object Structure

Operational Space

VODM governs the space between:

a bounded Value Object

and

a differentiated Value Object structure suitable for downstream value governance.

This operational space includes:

object differentiation; object equivalence assessment; component differentiation; composite differentiation; capability differentiation; functional differentiation; relational differentiation; configuration differentiation; temporal differentiation; lifecycle differentiation; duplicate-object detection; object aggregation risk; object fragmentation risk; value-lifecycle independence; differentiation uncertainty; differentiation state; differentiation traceability; differentiation reassessment after material change.

Why Differentiation Matters

Two things may appear to be the same asset while possessing different value lifecycles.

Likewise, several things may appear separate while together constituting one meaningful Value Object.

Consider a manufacturing facility containing three production lines.

The facility may be governed as:

one integrated production capability

or the three lines may be governed as:

Value Object A — Production Line 1 Value Object B — Production Line 2 Value Object C — Production Line 3

Both approaches may be legitimate.

The correct differentiation depends upon what Value Object has been identified, how its boundary has been established, and whether the components possess materially independent value lifecycles.

Without explicit differentiation, later value analysis may silently mix:

component value; composite value; capability value; replacement value; operational value; relational value; value created through integration.

VODM makes these distinctions governable before value itself is analyzed.

Differentiation Principle

A Value Object should be differentiated where a material distinction exists that would cause its value lifecycle to be governed differently from another object or component.

A material distinction may arise through:

different constitution; different function; different capability; different relationship; different configuration; different lifecycle; different dependency structure; different temporal existence; different value-origin relationship; different value-emergence conditions; different degradation pathway; different restoration possibility; different extinction condition.

The governing question is not:

Do these objects look different?

It is:

Would treating them as the same Value Object materially distort governance of their value lifecycles?

Similarity ≠ Identity

Two Value Objects may be highly similar without being the same governed object.

Examples include:

two identical machines; two identical software licenses; two apartments in the same building; two units of inventory; two identical datasets; two equivalent production lines; two contractual rights with identical terms.

Similarity may support comparison.

It does not erase object distinction.

Therefore:

Similarity ≠ Value Object Identity

Difference ≠ Separate Value Object

Conversely, the existence of differences does not automatically require separate Value Objects.

An integrated industrial system may contain:

different machines; different software components; different operational processes; different infrastructure elements.

Those differences may remain internal to one composite Value Object where the governed object is the integrated operational capability.

Therefore:

Internal Difference $\neq$ Automatic Object Separation

Differentiation Candidate

A Differentiation Candidate is:

an object, component, capability, relationship, configuration, or other materially distinguishable element whose value lifecycle may require governance separately from another Value Object or from the composite object within which it exists.

Identification of a Differentiation Candidate does not automatically create a new Value Object.

It creates a requirement for differentiation assessment.

Material Difference Assessment

VODM assesses whether differences are material to the value lifecycle.

A difference may be material where it changes:

what value may originate from the object; when value may emerge; whether value may be independently recognized; the state of value; how value may change; how value may develop; how value may degrade; whether value may reach zero independently; whether value may be restored independently; whether value may become extinct independently.

This creates a powerful architectural test:

Can the relevant value lifecycle materially diverge?

If yes, separate Value Object governance may be required.

Value-Lifecycle Independence Test

VODM introduces the Value-Lifecycle Independence Test.

The test asks whether a candidate can possess a materially independent lifecycle across one or more VFM® stages.

For example:

Can it have its own:

Value Origin?

Can value emerge from it independently?

Can its value state differ from the surrounding object?

Can it develop while the surrounding object degrades?

Can it reach zero while the composite retains value?

Can it be restored independently?

Can its value become extinct while another related object continues to possess value?

The greater the lifecycle independence, the stronger the basis for differentiation.

Object Differentiation

Object differentiation separates distinct Value Objects that may otherwise be treated as one.

Example:

A company owns two physically identical warehouses.

They may share:

design; purpose; ownership; accounting category.

But they occupy different locations, have different dependencies, different operational conditions, and potentially different value lifecycles.

VODM preserves them as distinct Value Objects where required.

Component Differentiation

A component within a composite Value Object may require independent differentiation.

For example:

Aircraft

may constitute one composite Value Object.

But:

Engine A

and

Engine B

may each possess independently relevant value lifecycles.

VODM can therefore establish:

Aircraft — Composite Value Object

with nested:

Engine A — Component Value Object Engine B — Component Value Object

without dissolving the aircraft as the composite Value Object.

Composite Differentiation

Multiple components may together produce a value phenomenon that does not exist at the component level.

Consider:

Land + Building + Machinery + Software + Workforce Capability + Process Integration

Each element may possess independent value.

But their integration may create:

Operational Manufacturing Capability

with a separate value lifecycle.

VODM therefore recognizes that:

Component Value Objects

and

Composite Value Objects

may legitimately coexist.

Composite Value $\neq$ Component Value Sum

A composite object may possess value arising from integration.

This means:

Composite Value $\neq$ necessarily Σ Component Values

Integration may create:

operational capability; network effects; interoperability; coordination capability; scale effects; access; strategic capability; productive capacity.

Conversely, dysfunctional integration may reduce the value of the composite even where individual components retain substantial value.

VODM does not measure this value.

It ensures the composite and its components can be governed separately where their value lifecycles differ.

Functional Differentiation

One physical or digital object may support several functions.

Those functions may create distinct value relationships.

For example, a building may function as:

production space; storage; office space; rental property; collateral; redevelopment opportunity.

VODM determines whether these functions merely affect one Value Object or whether one or more functions constitute independently governable Value Objects or capabilities.

The distinction must be explicit.

Capability Differentiation

Capabilities may require differentiation from the physical objects that enable them.

For example:

Machine

and

Production Capability

are not automatically the same Value Object.

A machine may retain resale value even where the production capability has collapsed.

Likewise, a production capability may remain valuable after individual machines are replaced.

Therefore:

Enabling Asset $\neq$ Capability Value Object

where their value lifecycles are materially independent.

Relational Differentiation

Value may relate to a relationship rather than exclusively to either participating object.

Examples include:

supplier relationship; distribution relationship; customer network; platform access; partnership; licensing relationship; contractual access; interoperability relationship.

VODM may differentiate:

Party A

from

Party B

from

Relationship AB

where each represents a materially distinct Value Object or value-bearing phenomenon.

This prevents relational value from being incorrectly assigned entirely to one participant.

Configuration Differentiation

The same underlying object may possess materially different value characteristics under different configurations.

Examples include:

machine configuration; software configuration; AI model configuration; infrastructure topology; production configuration; service bundle; portfolio composition.

VODM determines whether configuration change merely modifies the same Value Object or creates a sufficiently distinct Value Object configuration requiring separate governance.

Version Differentiation

Versions may need to remain distinguishable where value-relevant characteristics materially differ.

For example:

Software Version 4.1

and

Software Version 7.0

may share identity continuity while possessing different:

capabilities; dependencies; compatibility; market relevance; operational utility; lifecycle position.

VODM may therefore preserve version-specific Value Object differentiation without redefining identity governance.

Temporal Differentiation

A Value Object may materially change across time.

The same physical object may represent different governed Value Object states or configurations at different moments.

For example:

Industrial Facility — 2024 Configuration

and

Industrial Facility — 2026 Configuration

may require differentiation where expansion, automation, restructuring, or capability changes materially alter the value lifecycle.

VODM therefore supports temporally differentiated Value Objects where required.

Value Object ≠ Legal Entity

A legal entity may contain many Value Objects.

A Value Object may also span multiple legal entities.

Therefore:

Legal Entity Boundary ≠ Value Object Differentiation Rule

VODM does not determine corporate identity, legal ownership, or legal consolidation.

Value Object ≠ Accounting Unit

Accounting systems may aggregate or separate assets according to accounting requirements.

VODM differentiates Value Objects according to the methodological requirements of value lifecycle governance.

Therefore:

Accounting Unit ≠ Value Object

and:

Accounting Separation ≠ Automatic Value Object Differentiation

Value Object ≠ Market Product

A product sold as one commercial unit may contain multiple Value Objects.

Likewise, several products may depend upon one common Value Object.

Commercial packaging therefore does not determine methodological differentiation.

Differentiation and Dependencies

Objects may share dependencies without becoming one Value Object.

For example:

two factories may depend upon the same:

energy grid; transport network; software platform; supplier; workforce pool.

Shared dependency does not erase object differentiation.

Therefore:

Shared Dependency $\neq$ Shared Value Object

Differentiation and Ownership

Common ownership does not merge Value Objects.

Different ownership does not automatically separate a composite Value Object.

Ownership may be relevant contextual information, but ownership governance belongs outside VODM.

Therefore:

Ownership Structure $\neq$ Value Object Differentiation

Differentiation and Value Origin

VODM does not determine Value Origin.

However, different value-origin relationships may indicate that differentiation is necessary.

If two components derive value through materially different origins, their value lifecycles may need to remain separately governable.

The actual governance of Value Origin belongs to the Value Origin Standard.

Differentiation and Value State

Two components of one composite object may simultaneously occupy different Value States.

For example:

Component A — positive value Component B — degraded value Component C — zero-value condition

while:

Composite Object — positive value

This is possible only if object differentiation is explicit.

VODM therefore creates the structural basis necessary for later Value State governance.

Differentiation and DeMonet™

A DeMonet™ transition may affect one differentiated Value Object without nullifying another.

For example:

a technology component may lose all admissible value while the broader system retains value through replacement, adaptation, or alternative capability.

VODM ensures that value nullification is applied to the correct Value Object.

Differentiation and ZVA™ — Zero Value Asset™

A ZVA™ — Zero Value Asset™ state belongs to a defined Value Object.

Without differentiation, a zero-value condition affecting one component could be incorrectly generalized to: the entire composite object; related objects; the owner; the surrounding system.

VODM prevents this category error.

Differentiation and ReMonet™

ReMonet™ may restore value to:

one component; one capability; one relationship; one configuration; the complete composite object.

VODM provides the object structure necessary to identify exactly what is being restored.

Differentiation State

VODM may assign a governed differentiation state.

1. Undifferentiated

Potential distinctions exist but have not yet been assessed.

2. Differentiation Candidate

One or more potential independent Value Objects have been identified.

3. Conditionally Differentiated

Separate governance is required only under defined conditions, purposes, configurations, or temporal contexts.

4. Differentiated

The relevant Value Objects or components are sufficiently distinguished for downstream value governance.

5. Integrated

The assessed elements remain governed as one composite Value Object despite internal differences.

6. Differentiation Under Review

Material change or unresolved uncertainty requires reassessment.

7. Superseded Differentiation

A previous differentiation structure has been replaced while remaining preserved for historical traceability.

Differentiation State ≠ Value State

A Value Object may be:

Differentiated

while its value state remains unknown.

Likewise:

Integrated

does not mean valuable.

Differentiation states govern methodological object structure.

Value States govern the condition of value.

Over-Differentiation Risk

Excessive differentiation may fragment one meaningful Value Object into artificial pieces.

This can obscure:

integration value; network effects; system capability; relational value; composite performance; coordinated value creation.

VODM therefore requires differentiation to have a material value-lifecycle basis.

Under-Differentiation Risk

Insufficient differentiation may combine materially different Value Objects.

This can conceal:

independent value states; degradation; zero-value conditions; restoration potential; lifecycle differences; component-level extinction; independent value origins.

VODM therefore governs both:

over-differentiation

and

under-differentiation.

VODM Boundary with VOIM

VOIM identifies the Value Object.

VODM determines whether identified or bounded objects must remain methodologically distinct from other objects or components.

Therefore:

Identification $\neq$ Differentiation

VODM Boundary with VOBM

VOBM establishes what belongs inside or outside a Value Object Boundary.

VODM establishes whether elements inside, between, or adjacent to those boundaries require separate Value Object governance.

Therefore:

Boundary ≠ Differentiation

A component may be inside a composite boundary and simultaneously be a differentiated Value Object.

VODM Boundary with Value Context

VODM determines object separation.

The subsequent context function determines the conditions under which the differentiated object enters value analysis.

Therefore:

Object Differentiation ≠ Value Context

VODM Boundary with CLA™

VODM does not establish a universal classification system.

It distinguishes Value Objects only to the extent necessary for value lifecycle governance.

CLA™ governs formal classification structures, criteria, decisions, lifecycle, registry, and assurance.

Therefore:

Differentiation ≠ Classification

VODM Boundary with OBIDENTITY®

VODM does not determine:

identity ownership; legal ownership; authority; permission; provenance; transfer rights.

OBIDENTITY® governs those spaces.

A single identity may contain multiple Value Objects.

Multiple identities may participate in one relational or composite Value Object.

VODM Boundary with GOA™

GOA™ governs governance structure and governance boundaries.

VODM governs differentiation of Value Objects for value lifecycle purposes.

Therefore:

Governance Separation ≠ Value Object Differentiation

VODM Boundary with ORA™

ORA™ governs Operational Reality.

VODM may use operational distinctions as evidence that Value Objects differ, but it does not govern operational execution or operational-state separation.

VODM Boundary with INTEGROS®

INTEGROS® governs integrity.

VODM does not determine whether a differentiated object possesses integrity.

It determines only whether the object should remain separately governable within the value lifecycle.

VODM Boundary with VALIDOS®

VODM establishes methodological differentiation.

VALIDOS® may validate claims concerning whether a differentiation basis is sufficiently supported where formal validation is required.

Therefore:

Differentiation Determination $\neq$ Validation Determination

VODM Boundary with ADIT®

ADIT® may provide evidence concerning components, configurations, relationships, historical changes, or other facts relevant to differentiation.

VODM governs what those distinctions mean for Value Object separation.

Evidence of Difference $\neq$ Differentiation Governance

Minimum Implementation Framework

A minimum VODM implementation should establish:

1. Bounded Value Object Input

Receive the bounded Value Object from VOBM.

2. Differentiation Purpose

Define why differentiation is required.

3. Differentiation Candidates

Identify objects, components, capabilities, relationships, functions, or configurations potentially requiring separate governance.

4. Material Difference Assessment

Determine whether differences are material to the value lifecycle.

5. Value-Lifecycle Independence Test

Assess whether candidate objects may possess materially independent value origins, states, transitions, degradation, restoration, or extinction.

6. Composite / Component Assessment

Determine whether component and composite Value Objects must coexist.

7. Relational & Capability Assessment

Determine whether relationships or capabilities constitute independently governable Value Objects.

8. Configuration & Temporal Assessment

Determine whether configuration or time creates materially distinct Value Objects.

9. Differentiation Determination

Establish which objects remain integrated and which must remain distinct.

10. Differentiation State

Assign a governed differentiation state.

11. Differentiation Record

Document the basis, conditions, uncertainty, relationships, and resulting Value Object structure.

Governance Outputs

VODM may produce:

Value Object Differentiation Record Value Object Differentiation Statement Differentiation Candidate Register Material Difference Assessment Value-Lifecycle Independence Assessment Component Differentiation Record Composite Value Object Structure Capability Differentiation Record Relational Differentiation Record Configuration Differentiation Record Temporal Differentiation Record Integrated Value Object Determination Differentiation Uncertainty Record Value Object Differentiation State Differentiated Value Object Structure

These outputs provide the governed object structure required for the remaining Value Identification functions and subsequent VFM® standards.

Enterprise Application

VODM addresses a fundamental problem in valuation, accounting, taxation, investment, asset management, public finance, infrastructure, digital systems, and economic analysis:

before value can be governed, it must be clear whether analysts are talking about one Value Object or several different Value Objects.

Consider an industrial site containing:

Land + Building + Machinery + Software + Intellectual Property + Production Capability + Energy Access + Customer Relationships

A conventional analysis may treat all of these as one asset.

Another may treat each separately.

A third may recognize both individual component value and an additional integrated production capability.

VODM provides a methodology for deciding which distinctions must remain explicit.

This becomes particularly important when:

one component degrades while others remain valuable; one capability reaches zero value; one relationship disappears; a component can be sold independently; integration creates additional value; a configuration change creates a new capability; value is restored only to part of the system; one Value Object becomes extinct while another survives.

Without differentiation, later value governance may assign a value event to the wrong object.

VODM therefore establishes the methodological separation necessary before VFM® begins governing the origin, emergence, recognition, state, change, development, degradation, nullification, restoration, and extinction of value.

Foundational Principles

Value governance requires knowing not only what the Value Object is, but what must remain distinct from it.

Similarity does not establish identity.

Difference does not automatically require separation.

Shared ownership does not create one Value Object.

Shared dependency does not create one Value Object.

A component may belong to a composite Value Object while simultaneously possessing its own value lifecycle.

A composite Value Object may possess value that does not exist independently in its components.

Value-lifecycle independence is a primary basis for Value Object differentiation.

Over-differentiation can destroy the visibility of composite value.

Under-differentiation can conceal independent value states and transitions.

Differentiation is not classification.

Differentiation State is not Value State.

A zero-value condition must attach to the correct Value Object.

Value restoration must identify exactly which Value Object is being restored.

Canonical Closing Statement

Value Object Differentiation Module establishes the third methodological control point of the Value Identification Standard by determining which bounded objects, components, capabilities, relationships, functions, configurations, and temporally distinct forms must remain separate for their value lifecycles to be governed consistently. Through governance of material difference, Value-Lifecycle Independence, component and composite structures, functional and capability distinctions, relational objects, configuration and temporal differentiation, integration, uncertainty, and differentiation states, VODM prevents materially different Value Objects from being collapsed into one analytical object while preventing one meaningful Value Object from being fragmented without methodological justification. By separating differentiation from identity, ownership, classification, validation, integrity, audit, operational reality, and value state, VODM establishes the object structure upon which subsequent VFM® governance can operate. Before value origin, emergence, recognition, state, change, development, degradation, nullification, restoration, or extinction can be attributed correctly, it must first be clear whether the value lifecycle belongs to one Value Object, one of its components, a relationship, a capability, a configuration, or the composite object created through their integration.

Modul4

Governed Links

[→ Value Identification Standard \(VIS\)](#)

[→ VFM® Architecture Map](#)

[→ Previous: Value Object Boundary Module \(VOBM\)](#)

[→ Next: Value Context Module \(VCM\)](#)

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>